



Distributed Drone Network for search and rescue at sea

A. di Cataldo^{1,*} and A. Zobeles^{2,*}

¹ Graduate Student in Mechatronic Engineering, MAT: 249426

² Graduate Student in Mechatronic Engineering, MAT: 249893

Abstract

The rapid proliferation of drone technologies in recent years has enabled a wide range of new applications in cooperative sensing, mapping, and navigation. This report investigates how different amounts of drones can exchange information to localize themselves, identify landmarks, construct a shared map of the environment, and explore the map in different cooperative ways. We examine both the case in which all drones possess equally accurate proprioceptive and exteroceptive sensors, and the scenario in which sensing precision varies across agents. For map obstacles positioning, we employ a modified weighted least squares (Maximum Likelihood) algorithm, tailored to grid-based environments. Additional methods used include Covariance Intersection for information fusion under intermittent communication constraints, the Extended Kalman Filter for self positioning, Metropolis–Hastings consensus and row-stochastic weighting strategies for distributed agreement, centralized fusion for cooperative localization of critical map features, and the use of the trilateration-based map alignment to orient their local reference frames to the global coordinate system. Once the target is detected the A* algorithm computes an optimal route for rescue units. Together, these techniques provide a robust framework for collaborative autonomous mapping in environments with sparse shared landmarks.

Submitted: 30 March 2026

Vol. 1, No. 1, 2025.



*Corresponding author:

✉ A. di Cataldo
andrea.dicataldo@studenti.unitn.it

✉ A. Zobeles
alessio.zobeles@studenti.unitn.it

1 Introduction

Localization in remote or not well-known areas has always been a problem due to the fact that accessing and exploring these areas is difficult and a process that requires a lot of time for humans. For this issue, drones can offer an agile and fast solution. So in this paper, we propose a distributed algorithm that allows the drones to efficiently map and explore with limited knowledge of the area. Several advantages of this application are:

- Fast mapping: with more drones, a bigger area can be explored in a faster way.
- High accuracy in the mapping: with the drones sensors, maps can be drawn precisely.
- Versatility: flying drones can be deployed practically everywhere.

1.1 Problem Formulation

In this simulation environment, the two central challenges are *self-localization* and *mapping* (SLAM). The drones begin at random positions at sea with a magnetometer, and possess a priori knowledge about the position of the landmarks, which they recognize in the map whenever one is in range. Their mission is twofold: to locate a *Target*—such as a grounded vessel or a person in distress—and to generate a safe route from the rescuers' initial location to the victim.

Because the drones have a restricted communication range, they can exchange information only during occasional encounters (they are loosely coupled). During these brief interactions, they share the essential data: their estimated positions and their local maps (see Fig. 1). To estimate and refine their own trajectories, each drone runs an *Extended Kalman Filter* (EKF). For global reference frame alignment upon landmark detection, the system utilizes the implemented *trilateration* algorithm. During initial encounters, when cross-covariances among agents

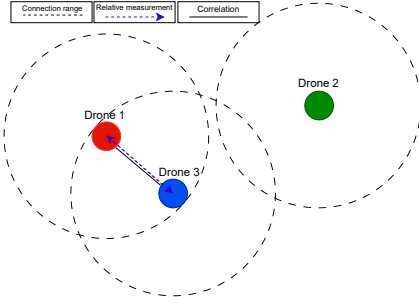


Figure 1. Loosely coupled drones connection scheme.

are unknown because covariance updates cannot be exchanged continuously, the *Covariance Intersection* (CI) method is applied for *Cooperative Localization* (CL) [1, 4].

To estimate what each drone locally observes, a modified *Weighted Least Square* (WLS) algorithm is employed. Once the victim is located and stable communication is maintained, a more precise and globally consistent reconstruction of the map is obtained by adopting a *centralized EKF fusion strategy* [3] which accounts for the correlation between drones estimates. When merging the individual maps, consensus is achieved using either the *Metropolis Hastings weight matrix* or a generic *row-stochastic matrix*, depending on the drones' measurement uncertainties. Finally, to compute an optimal path from the starting point to the victim, the *A* algorithm* [5] is implemented.

1.2 Paper Organization

The rest of the report is organized as follows. Section 2 describes the models and sensors used in our simulation. Section 3 details the distributed solutions and algorithms we developed. Section 4 explains the details of the implementations of the adopted solutions. Section 5 presents the simulation results of the case scenario in which a drone has better sensors and different amounts of drones and search algorithms are tested, and finally, Section 6 concludes the report with the discussion and possible future implementations.

2 Adopted Models

2.1 Sensors and Communication System

Each drone starts with the knowledge of where the landmarks are. In order for them to estimate the relative position from their position to the landmark or other entities, the drones have a radar with a standard deviation of σ_{radar} , which has a limited range, a camera to map the environment, an Ultrasonic Altimeter to control the drones altitude (which is always the same

cause we're at sea), and a WiFi, which also has a limited connection range that allows the drones to share information. These values (σ_{radar} , the WiFi connection range, and the camera max view area) are treated as variables at the start of the code to be able to simulate various scenarios. For example, if there are drones that can take less noisy measurements than others.

2.2 System Model/Dynamics

To model the drone dynamics, we assume a simple linear, orientation-free model. Each drone is characterized by its position in the $x-y$ plane. Based on its velocity and direction, the state is updated accordingly. Since the simulation environment is finite, control mechanisms and position constraints are introduced to prevent the drone from exiting the boundaries.

The state-update equation is

$$s_{D,k+1} = f(s_{D,k}, u_{D,k}, \nu_{D,k}) \quad (1)$$

where $s_{D,k}$ denotes the x and y coordinates of drone D at time step k . The input $u_{D,k}$ represents the velocity input, decomposed into its x and y components. The term $\nu_{D,k}$ is an additive Gaussian noise that embeds both the uncertainty on the model and the control, with $\nu_{D,k} \sim \mathcal{N}(0_2, Q)$, where Q is the covariance matrix computed from the velocity standard deviation $\sigma_{velocity}$ [m/s], treated as a simulation parameter similarly to σ_{radar} [m], which is the sensor that the drones use to take measurements with a Gaussian noise:

$$z_k = z_{true}(1 + \epsilon), \epsilon \sim \mathcal{N}(0, \sigma_{radar}^2) \quad (2)$$

The sensor has a maximum range z_{max} at which the radar is able to take a measurement.

For linear simplification, the motor dynamic response is neglected. Thus, when the drone changes direction, its velocity magnitude remains unchanged, and the Gaussian noise accounts for deviations from ideal motion (e.g., wind-induced drift, small orientation errors, etc.). This velocity is also considered a system variable, although it does not introduce any special additional effect in the simulation except for faster movement.

3 Solution

3.1 Simulation scenario

The simulation environment is randomly generated each time, and to each category of obstacle or terrain

feature is assigned a specific value. In a tunable way, it can be set how many obstacles in % have a unique ID, which is what the drones, whenever one of these IDs (landmark) is found, will use to estimate their relative position. The more IDs are present, the easier it will be for the drones to find an ID to correct their estimate. Then the simulation consists of the following entities:

- **Environment:** each simulation occurs in a 1000x1000 predefined square map.
- **Features:** static environmental elements, including coastline, buoys, and reefs, represent navigational obstacles for marine vessels.
- **Landmarks:** a variable quantity of features with a priori given coordinate $s_{L,i} = [x_{L,i} y_{L,i}]$.
- **Target:** a static entity that could represent anything that the drones need to find, and is positioned randomly on any obstacle that is neither the coast nor the open sea.
- **Drones:** are moving units that have the task to explore and map the environment, they are represented as a circular moving dot. And each robot is identified by an x and y coordinate, which refer to the center of this encumbrance.

The simulation begins with the drones placed at random locations, ensuring a minimum distance from the target to prevent the simulation from ending prematurely. When a drone detects the target and its associated uncertainty falls below a specified threshold, it emits a visual signal (such as a flare), prompting the other drones to navigate toward that position. Once all drones have reached the location, they follow the lead drone (drone 1) at a fixed distance and proceed toward the point where the rescuers begin their mission. After the drones arrive at this destination, the fastest and safest path that avoids obstacles and guides the rescue boat to the target is computed.

3.2 Self localisation via Extended Kalman Filter

The state estimation of each drone is obtained by predicting and estimating its position through the Extended Kalman Filter. In particular, the drone computes a prediction over its position starting from its previous and its dynamic (speed, time interval and direction), then if the drone detects a landmark, it performs an update combining the relative measurement from the landmark, and its estimate coming from its dynamic. The Extended Kalman Filter equations are:

$$\hat{s}_{k+1}^- = f(\hat{s}_k, \mathbf{u}_k) \quad (3)$$

$$F_k = \left. \frac{\partial f(s, u, \nu)}{\partial \mathbf{s}} \right|_{\mathbf{s}=\hat{s}_k, \mathbf{u}=\mathbf{u}_k, \nu=0} \quad (4)$$

$$P_{k+1} = F_k P_k F_k^\top + G_k Q G_k^\top \quad (5)$$

$$G_k = \left. \frac{\partial f(s, u, \nu)}{\partial \nu} \right|_{\mathbf{s}=\hat{s}_k, \mathbf{u}=\mathbf{u}_k, \nu=0} \quad (6)$$

$$(7)$$

with P being the state covariance matrix for all drones, where the variance of each state variable of each drone is found at index $ij = D - 1, \forall j = i, D \geq 1$ for the x component, and at index $ij = D \cdot 2, \forall j = i, D \geq 1$ for the y , then in the off-diagonal terms, the covariances between states and drones. F_k and G_k are the Jacobian matrices of the dynamics and process noise respectively. Q is the process noise covariance matrix:

$$Q = \begin{bmatrix} \sigma_{velocity_{x,1}}^2 & 0 & \dots & 0 \\ 0 & \sigma_{velocity_{y,1}}^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_{velocity_{x,D}}^2 & 0 \\ 0 & 0 & \dots & \sigma_{velocity_{y,D}}^2 \end{bmatrix} \quad (8)$$

, and at state $k = 1$ P_0 and $\hat{s}_0$ being respectively the initial state covariance matrix representing the initial uncertainty over the state, and the initial estimate of the position for each drone. Then, if the drones measure a landmark (or more), the update in the position and state covariance matrix becomes:

$$H_{k+1} = \left. \frac{\partial h(s, \epsilon)}{\partial \mathbf{s}} \right|_{\mathbf{s}=\hat{s}_k, \epsilon=0} \quad (9)$$

$$y_{k+1} = z_k - h(\hat{s}_{k+1}, s_{L,i}, 0) \quad (10)$$

$$S_{k+1} = H_{k+1} P_{k+1}^- H_{k+1}^\top + R_k \quad (11)$$

$$K_{k+1} = P_{k+1}^- H_{k+1}^\top S_{k+1}^{-1} \quad (12)$$

$$\hat{s}_{k+1} = \hat{s}_{k+1}^- + K_{k+1} y_{k+1} \quad (13)$$

$$P_{k+1} = (I - K_{k+1} H_{k+1}) P_{k+1}^- \quad (14)$$

being S the covariance matrix of the innovation, R the measurement noise covariance matrix, z_k the measurement affected by the white noise with standard deviation σ_{radar} , $h(\hat{s}_{k+1}, s_{L,i})$ the expected measurement at time step k based on the predicted

state $\hat{s}_{k+1}$, which is computed using all measurements up to time k . In our model, F and G correspond to the identity matrix I , and $h(\hat{s}_{k+1}, s_{L,i}, 0)$ corresponds to the Euclidean distance formula in 2D:

$$h(\hat{s}_{k+1}, s_{L,i}, 0) = \sqrt{(x_L - \hat{x}_{k+1}^-)^2 + (y_L - \hat{y}_{k+1}^-)^2} \quad (15)$$

and H_k to:

$$\begin{bmatrix} \frac{\hat{x}_{k+1}^- - x_L}{\sqrt{(x_L - \hat{x}_{k+1}^-)^2 + (y_L - \hat{y}_{k+1}^-)^2}} \dots \\ \dots \frac{\hat{y}_{k+1}^- - y_L}{\sqrt{(x_L - \hat{x}_{k+1}^-)^2 + (y_L - \hat{y}_{k+1}^-)^2}} \end{bmatrix} \quad (16)$$

3.3 Self localisation through other drones' relative measurements with the Covariance Intersection (CI) method

When a drone measures the position of another drone to refine its own estimate, the other drone cannot be treated as a fixed landmark, since its position is also uncertain. Treating it as a landmark would lead to an overconfident estimate. Furthermore, drones cannot continuously track their mutual correlation, because communication links may drop when the drones move too far apart.

The CI algorithm allows a drone to incorporate another drone's position estimate while accounting for its uncertainty, without requiring continuous correlation tracking.

In the CI method, drone A performs a relative measurement of drone B, denoted as $\hat{s}_{A \rightarrow B}$, and computes the updated state covariance $P_{A \rightarrow B}$ for B's estimated position. This covariance accounts for both A's prior uncertainty and the error introduced by the measurement itself. $P_{A \rightarrow B}$ is calculated as:

$$P_{A \rightarrow B, k+1} = \tilde{H}_{k+1}^{AB} P_{A, k+1} (\tilde{H}_{k+1}^{AB})^\top + R_k \quad (17)$$

A. Carrillo et al. [4] multiply the R_k matrix by a coordinate transformation. In our case, this step is unnecessary because when the drones detect each other, they align their coordinate frames, eliminating the need for further transformations upon subsequent encounters. To consistently fuse estimates with unknown correlations, the covariance update step is given by:

$$P_{B, k+1} = [\omega(P_{B, k})^{-1} + (1 - \omega)(P_{A \rightarrow B, k})^{-1}]^{-1} \quad (18)$$

where $\omega \in [0, 1]$ is chosen to minimize either the trace or the determinant of $P_{B, k+1}$. In this paper, following A. Carrillo [4], we also choose to minimize the trace. The resulting updated estimate for B is then:

$$\hat{s}_{B, k+1} = P_{B, k+1} [\omega(P_{B, k})^{-1} \hat{s}_{B, k} + \dots \quad (19)$$

$$\dots (1 - \omega)(P_{A \rightarrow B, k})^{-1} \hat{x}_{A \rightarrow B, k}] \quad (20)$$

The receiving drone then updates its own estimate, and if this update results in a covariance matrix P with reduced uncertainty, the drone's position estimate is also improved; otherwise, the drone uses only its own covariance matrix in the calculation of $P_{B, k+1}$ and the estimate doesn't change.

3.4 Mapping and recognition strategy using a modified WLS algorithm

At each time step k , every drone takes an image of all the cells within its visibility range. Since the map is static, we want the estimates from previous iterations to remain stable rather than becoming increasingly uncertain. The estimates should only be updated when new images are processed, allowing the uncertainty to progressively decrease and converge to the true value. To implement this, we maintain two maps: one representing the uncertainty of each cell fig:2, and another storing the information about what has been observed fig6.

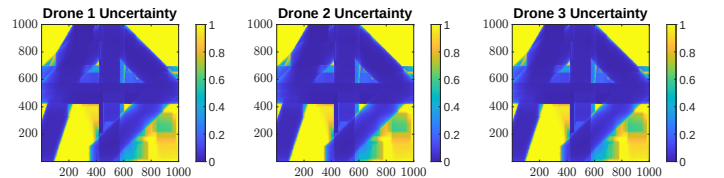


Figure 2. Uncertainty maps for three drones at the end of the exploration mission.

Then the algorithm for the map estimation is:

$$K_{k, map} = \frac{P_{k, map}}{P_{k, map} + R_{k, map}} \quad (21)$$

$$\hat{s}_{k+1, map} = \hat{s}_{k, map} + K_{k, map} (z - \hat{x}_{k, map}) \quad (22)$$

$$P_{k+1, map} = (1 - K_{k, map}) P_{k, map} \quad (23)$$

with z being the information of the seen cell. $P_{0, map} = 1$ is the initial uncertainty, meaning 1 no knowledge about that cell. Cause the positioning of the obstacles depends on the drone's estimate at $\hat{s}_{k+1}$ it is important to also account for this uncertainty. So in consequence, $R_{k, map}$ is modeled in such a way that the higher is the uncertainty on the drone state estimate, the less the drone uncertainty about this cell should decrease to

allow better measurements to be taken. $R_{k,map}$ is the calculated as:

$$R_{k,map} = P(xx_D) + P(yy_D) \quad (24)$$

which represents how uncertain the drone is about its estimate fig:5.

3.5 Map sharing through consensus

When a drone finds another, and could already estimate the other's relative pose, then they should reach a consensus on what they saw in common, and then share what one saw and the other did not. There are two cases:

- If the drones have all the same standard deviation, then we want the reached consensus to be the average. To reach this consensus, the Metropolis-Hastings weights are used. Even if all the drones are not always connected, while they are still searching for the target, when they find it, they all come together, then they are strongly connected, and the consensus can be reached. The Metropolis-Hastings weights for the Matrix W are calculated as:

$$w_{ij} = \begin{cases} \frac{1}{\max(d(i), d(j)) + 1}, & \text{if } (j, i) \in \mathcal{E} \text{ and } i \neq j, \\ 1 - \sum_{j=1, i \neq j}^n w_{ij}, & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

where $d(i)$ stands for the degree of the node (in our case, to how many other drones the i drone is connected to), and the first condition checks if the ij connection exists. The resulting matrix W is a doubly stochastic matrix where both row and column sums equal to one.

- If the drones don't have the same standard deviation, then we don't want the reached consensus to be the average, but a solution that is a weighted average with the highest weight to the drone with the smallest standard deviation. In this case, we use a row stochastic matrix, where the weights of the nodes are calculated depending on the standard deviation in both velocity and radar standard deviation. To build this matrix, the joint variance of each drone is defined as:

$$\sigma_{\text{joint},i}^2 = \left(\frac{\sigma_{\text{velocity},i}}{2} + \frac{\sigma_{\text{radar},i}}{2} \right)^2,$$

then the inverse values of the joint variance are used to construct a row-stochastic weight

matrix W . For each drone i , let $\mathcal{N}(i)$ denote the set of its neighbors. We define the augmented neighborhood as:

$$\tilde{\mathcal{N}}(i) = \mathcal{N}(i) \cup \{i\},$$

which includes the drone itself.

For each row i , the denominator for normalization is:

$$\text{denom}_i = \sum_{j \in \tilde{\mathcal{N}}(i)} \frac{1}{\sigma_{\text{joint},j}^2}.$$

If the denominator is nonzero, the weights assigned to drone i 's neighbors (and itself) are:

$$w_{ij} = \begin{cases} \frac{\frac{1}{\sigma_{\text{joint},j}^2}}{\text{denom}_i}, & \text{if } j \in \tilde{\mathcal{N}}(i), \\ 0, & \text{otherwise.} \end{cases}$$

which gives row sum equal to one. When this matrix is multiplied by a state vector m , which contains the map each drone created, and this operation is done for a sufficient number of times, the result stabilizes at the weighted average, and the consensus is reached in the form of:

$$Wm = m \quad (25)$$

to prove where m converges, the left eigenvector is calculated after the eigenvalues of W are calculated in the form:

$$(W - \lambda I)^T = 0 \quad (26)$$

Then, for the dominant eigenvalue, the equation is solved, and a corresponding eigenvector v is calculated and subsequently normalized, so when the equation:

$$\text{norm}(v)^T m = c \quad (27)$$

is solved, the consensus c is reached. Which in a row stochastic matrix, and a fully connected graph, c corresponds to the weighted average.

3.6 Cooperative Localisation

Cooperative localisation is the process through which our team of drones estimates their absolute positions by exploiting not only their own motion and sensor data but also relative measurements exchanged with neighboring agents. Even when individual localisation is affected by noise or drift, sharing local information

enables the swarm to obtain a more accurate and globally consistent estimate of the state.

In contrast to standard localisation, where each drone relies solely on its own sensors, cooperative localisation couples each agent's estimate with those of its neighbours. This coupling is formalised through the joint state vector:

$$\mathbf{s} = [\mathbf{s}_1^\top \quad \mathbf{s}_2^\top \quad \cdots \quad \mathbf{s}_N^\top]^\top,$$

where $\mathbf{s}_i \in \mathbb{R}^2$ represents the (x, y) position of drone i . The associated covariance matrix is

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_{11} & \mathbf{P}_{12} & \cdots & \mathbf{P}_{1N} \\ \mathbf{P}_{21} & \mathbf{P}_{22} & \cdots & \mathbf{P}_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{N1} & \mathbf{P}_{N2} & \cdots & \mathbf{P}_{NN} \end{bmatrix},$$

where $\mathbf{P}_{ii}$ denotes the uncertainty of drone i 's pose, while $\mathbf{P}_{ij}$ with $i \neq j$ encodes the cross-covariance between drones i and j . These cross-covariances are fundamental because they are the channel through which information spreads across the swarm: a single relative measurement between drones a and b can indirectly refine the estimates of all other correlated agents.

Whenever two drones are within sensing range, they can measure their relative distance using onboard sensors. A typical relative measurement at time k is:

$$z_{ab,k} = h(\mathbf{s}_{a,k}, \mathbf{s}_{b,k}) + \nu_k,$$

where $z_{ab,k}$ is the noisy measurement, ν_k is the zero-mean Gaussian noise covariance $\mathbf{R}_k$, and $h(\cdot)$ is the nonlinear measurement function. In our system, h corresponds to the **Euclidean distance** between two drones.

Before incorporating such a measurement, each drone predicts its state using its motion model. This results in the standard EKF prediction:

$$\hat{\mathbf{s}}_{k+1}^- = f(\hat{\mathbf{s}}_k, \mathbf{u}_k),$$

$$\mathbf{P}_{k+1}^- = \mathbf{F}_k \mathbf{P}_k \mathbf{F}_k^\top + \mathbf{G}_k \mathbf{Q}_k \mathbf{G}_k^\top.$$

where f is the dynamic model, $\mathbf{F}_k$ its Jacobian with respect to the state, $\mathbf{G}_k$ the Jacobian with respect to the process noise, and $\mathbf{Q}_k$ the corresponding noise covariance. This step typically increases uncertainty due to motion and unmodeled disturbances.

When drone a detects drone b , the filter performs a cooperative update. The innovation is defined as:

$$\mathbf{r}_{ab,k} = z_{ab,k} - h(\hat{\mathbf{s}}_{a,k}^-, \hat{\mathbf{s}}_{b,k}^-),$$

where r_{ab} is the predicted inter-drone distance, which quantifies the discrepancy between the predicted and measured relative distances. To relate this measurement to the global state, the EKF uses the Jacobians

$$\mathbf{H}_a = \frac{\partial h}{\partial \mathbf{x}_a} = \begin{bmatrix} \frac{x_a - x_b}{r_{ab}} & \frac{y_a - y_b}{r_{ab}} \end{bmatrix}, \quad \mathbf{H}_b = -\mathbf{H}_a,$$

These are then embedded into the global measurement Jacobian $\mathbf{H}_{k+1}$, ensuring that only the columns corresponding to drones a and b are affected.

Once the Jacobians are defined, the filter computes the innovation covariance

$$\mathbf{S}_{k+1} = \mathbf{H}_{k+1} \mathbf{P}_{k+1}^- \mathbf{H}_{k+1}^\top + \mathbf{R}_k,$$

which balances predicted uncertainty against measurement noise, and then the Kalman gain:

$$\mathbf{W}_{k+1} = \mathbf{P}_{k+1}^- \mathbf{H}_{k+1}^\top \mathbf{S}_{k+1}^{-1}.$$

The gain determines how strongly the measurement should influence the state estimate.

The cooperative update then refines the entire state:

$$\hat{\mathbf{s}}_{k+1} = \hat{\mathbf{s}}_{k+1}^- + \mathbf{W}_{k+1} \mathbf{r}_{ab,k}, \quad (28)$$

$$\mathbf{P}_{k+1} = \mathbf{P}_{k+1}^- - \mathbf{W}_{k+1} \mathbf{S}_{k+1} \mathbf{W}_{k+1}^\top. \quad (29)$$

Because $\mathbf{P}_{k+1}^-$ encodes cross-covariances, this update improves the estimates of all drones, not only of those directly involved in the measurement. As a result, the swarm maintains a coherent formation and can coordinate movements reliably even in challenging sensing conditions, enabling effective operation in complex environments such as search-and-rescue scenarios.

3.7 Trilateration

Trilateration constitutes a geometric positioning methodology that determines an object's coordinates through distance measurements from known reference points, termed anchors. Unlike triangulation which employs angular measurements, trilateration operates exclusively on range data. In a two-dimensional plane, each distance measurement d_i from an anchor situated at coordinates (x_i, y_i) establishes a circular locus of potential positions for the target object, described by the equation $(x - x_i)^2 + (y - y_i)^2 = d_i^2$. The intersection of two such circles typically yields two candidate solutions. The introduction of a third, non-collinear anchor resolves this ambiguity, producing a unique

solution at the common intersection point of all three circles.

When dealing with $n \geq 3$ anchors, the system of equations becomes overdetermined. The optimal position estimate is consequently obtained by solving a non-linear least squares optimization problem that minimizes the sum of squared residuals between measured and calculated distances:

$$\min_{x,y} \sum_{i=1}^n \left[\sqrt{(x-x_i)^2 + (y-y_i)^2} - d_i \right]^2. \quad (30)$$

This formulation is generally solved using iterative numerical methods such as Gauss-Newton or Levenberg-Marquardt algorithms, which require an initial position estimate. Practical implementations must contend with several error sources including sensor measurement noise, non-line-of-sight conditions that cause distance overestimation, and geometric dilution of precision resulting from poor anchor placement. The accuracy of the solution is highly dependent on the geometric configuration of anchors relative to the target, with optimal precision achieved when anchors surround the target in a non-collinear arrangement.

3.8 A* Algorithm

The A* algorithm is a widely used pathfinding method that efficiently finds the shortest path between two points while avoiding obstacles. In our SAR system, once the target has been located, the drones reach it and then head together to the boat. Once they arrive at the boat, the drones provide a map of what they have explored and the A* algorithm computes the optimal path from the rescue boat's starting position to the target location, ensuring the fastest and safest route for rescue operations.

The algorithm represents a synthesis of two fundamental graph traversal methodologies: Dijkstra's algorithm, which provides guarantees of path optimality through exhaustive exploration, and greedy best-first search, which enhances computational efficiency through the utilization of heuristic guidance. This hybrid approach enables A* to achieve both completeness and performance improvements in pathfinding tasks.

The algorithm's core mechanism relies on the maintenance and evaluation of two distinct cost functions for each node n within the search space:

- $g(n)$: actual cost from the start node to node n

- $h(n)$: heuristic estimate of the cost from node n to the target node

The total estimated cost is consequently given by:

$$f(n) = g(n) + h(n) \quad (31)$$

Given the critical role of the heuristic component in this formulation, the selection of an appropriate heuristic function $h(n)$ becomes paramount to the performance and correctness of the A* algorithm, as it directly influences both the optimality of the solution and the computational efficiency. The heuristic must satisfy two fundamental properties:

- **Admissibility:** A heuristic is considered admissible if it never overestimates the true cost to reach the target. Formally, this requires that $h(n) \leq h^*(n)$ for every node n in the search space, where $h^*(n)$ represents the actual minimal cost from node n to the target. This property guarantees that the algorithm will always find an optimal solution, should one exist.
- **Consistency:** A heuristic is consistent if the estimated cost from a node n to the target is no greater than the step cost to any successor m plus the estimated cost from m to the target. This is formalized by the triangle inequality $h(n) \leq c(n, m) + h(m)$ for all nodes n and their immediate neighbors m , where $c(n, m)$ is the cost of moving from n to m . Consistency ensures that the algorithm finds an optimal path without the need to re-expand nodes.

In our implementation, the **Euclidean distance** serves as the heuristic function. This choice is particularly suitable for our application, as it is provably both admissible and consistent within our 2D grid environment, which permits movement in eight directions.

With the heuristic function properly defined, the A* algorithm operates by systematically evaluating and expanding nodes based on the minimization of the cost function $f(n)$. The procedure is governed by the following mathematical framework and sequential steps, where the algorithm's decision-making is driven by four fundamental equations:

1. Initialization:

$$g(start) = 0 \quad (32)$$

$$f(start) = h(start) \quad (33)$$

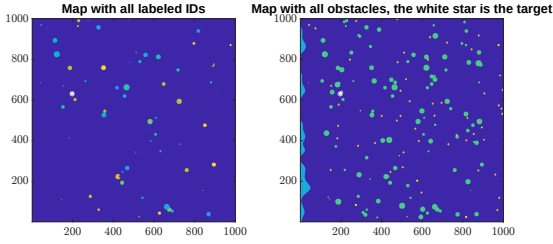


Figure 3. Map example with the position of where the landmarks are (left), and the full map with all obstacles and the target (right).

2. Node Evaluation:

$$f(n) = g(n) + h(n) \quad (34)$$

3. Cost Update (when a better path is found):

$$g(m) = g(n) + c(n, m) \quad (35)$$

where $c(n, m)$ represents the movement cost from node n to its neighbor m .

This structured approach ensures that the algorithm efficiently explores the most promising paths while guaranteeing optimality through systematic evaluation of the cost functions.

4 Details Implementation

4.1 Initialization and Environment Definition

When initializing the simulation, several parameters can be set and adjusted to test specific scenarios: the number of known landmarks (see Fig. 3), the standard deviation of each drone, the communication and visibility ranges, the initial position, the exploration method after the trilateration, and the relative positions of the other drones with respect to drone 1 when they rendezvous at the victim and proceed to the rescuers. For example, these settings allow testing situations such as one drone having better velocity calibration than another, a drone with reduced visibility, or varying numbers of landmarks to aid navigation.

At the start of exploration, the drones move in random directions until they find a Landmark to do the trilateration. Then they start to explore the map in a selected search algorithm.

4.2 Map exploration

Since the drones have to explore the map until the target is found, a suitable exploration algorithm is needed so that the drones can find the target in the fastest way possible. So three exploration strategies will be tested to find the best among these cases:

- **Uniform Boustrophedon decomposition:** The robots explore the area by performing a sequence of parallel, equally spaced straight-line motions along a chosen sweep direction, aligned with one of the coordinate axes. At the end of each sweep, the robot reverses its direction and proceeds along the adjacent line, creating a continuous back-and-forth lawnmower pattern that guarantees full coverage of the area. This deterministic motion minimizes redundant traversal and ensures full map covering, but at the cost that if the target is at the end of this lawnmower motion, the maximum possible time for N drones is taken.
- **Random space partition:** The drones divide the map into an $M \times M$ matrix. When one of these cells is fully explored, it becomes occupied, and the drones don't go there anymore, when the drones encounter, the drone with the highest identification number goes to explore to the farthest free cell of this matrix, and the nearest cell, which will be explored by the other drone, is also marked as explored to avoid the drone going back there.
- **Just Random:** The drones search for the nearest unexplored cell.

All these algorithms are used until the target is found.

4.3 Map recognition and distances estimation

To simulate realistic drone behavior, each drone observes its surroundings and updates the map at the location corresponding to its current estimated position. The visibility region of a drone located at (x, y) is defined as

$$\begin{aligned} x_{\text{vis}} &\in [x - v_{\text{range}}, x + v_{\text{range}}], \\ y_{\text{vis}} &\in [y - v_{\text{range}}, y + v_{\text{range}}]. \end{aligned}$$

Equivalently, the visible area can be written as the intersection of subsets of $\mathbb{R}^2$:

$$\begin{aligned} \mathcal{V}(x, y) &= \{(x', y') \in \mathbb{R}^2 \mid x' \in [x - v_{\text{range}}, x + v_{\text{range}}]\} \\ &\cap \{(x', y') \in \mathbb{R}^2 \mid y' \in [y - v_{\text{range}}, y + v_{\text{range}}]\}. \end{aligned}$$

with v_{range} as the maximum distance at which they can detect obstacles.

4.4 Implementation of Trilateration

This project implements a sequential trilateration method to correct positional drift in GPS-denied

drones. The approach uses a single visual anchor and multiple drone positions over time, creating virtual anchors from the drone's trajectory to enable self-correction.

The algorithm begins by detecting visible landmarks. If no anchor is active, the closest landmark becomes the exclusive reference for the entire calibration cycle, preventing switching errors. Calibration triggers conservatively only when the drone enters a trusted zone within half the sensor range, ensuring reliable measurements.

During calibration, the drone executes a four-step movement pattern while recording: its estimated position $\mathbf{P}_k$, the distance d_k to the anchor, and the actual movement $\mathbf{u}_k$ between steps. This produces the dataset $\{\mathbf{P}_k, d_k, \mathbf{u}_k\}_{k=1}^4$ for optimization.

The system then solves a constrained optimization problem balancing trajectory consistency with distance measurements:

$$\min_{\mathbf{X}} \sum_{k=1}^3 \|\mathbf{P}_k + \mathbf{u}_k - \mathbf{P}_{k+1}\|^2 + \lambda \sum_{k=1}^4 (\|\mathbf{P}_k - \mathbf{A}\| - d_k)^2, \quad (36)$$

where $\mathbf{X} = [x_1, y_1, \dots, x_4, y_4]^T$ represents optimized poses, $\mathbf{A}$ is the anchor position, and λ weights motion versus measurement fidelity. This sparse nonlinear least-squares problem is solved efficiently using standard numerical methods.

Correction is applied directly to the drone's perceptual model. The drift vector $\mathbf{t} = \mathbf{P}_4^{\text{est}} - \mathbf{x}_k$ is computed from the optimized final position and rounded to grid resolution. The local occupancy maps are shifted by $\mathbf{t}$ using a circular shift operation, realigning the drone's internal representation with the global anchor frame. The state estimate and covariance are then updated, completing the calibration cycle.

The system's convergence is guaranteed by Markov chain principles. The steady-state distribution π , derived from the transition matrix eigenvector for eigenvalue 1, ensures $\lim_{k \rightarrow \infty} \mathbf{x}_k = (\pi^T \mathbf{x}_0) \cdot \mathbf{1}$, providing theoretical stability guarantees for long-term operation.

4.5 Communication and localisation between drones

The mission of the drone team is twofold: first, they must locate the victim; then, they must accurately map the path from the victim to the rescuer's starting position. This detailed map allows the rescuer to plan

an optimal route while being aware of all obstacles lying between the start point and the target.

The motion model used for the drones includes significant uncertainty: the standard deviation of the velocity noise can reach up to 60% of the maximum velocity. This is because Gaussian noise is assumed, and approximately 99.7% of the disturbances lie within a 3σ interval. Given this level of uncertainty, it is essential that, during the mapping phase, each drone can benefit from the measurements collected by the others.

To achieve this, the system uses the *Extended Kalman Filter with Centralized Fusion*, described in Chapter 3.6. In this approach, a central fusion node collects all available measurements and state predictions from the drones, performs a global update, and sends the corrected estimates back to the team. For this to operate correctly, the drones must maintain a *connected communication graph* at all times so that every drone can forward its data (directly or indirectly) to the central fusion node. In contrast to when they are still looking for the target that the drones connect only sporadically, and the information is merged through the CI method.

To ensure both wide environmental coverage and continuous connectivity, after every drone has reached the target, they are coordinated as follows: Drone 1 is commanded to move toward the start position, while the others maintain a fixed relative distance from Drone 1, positioned respectively above and below its trajectory. At the beginning of this formation, Drone 1 waits for a variable amount of time to allow the other Drones to reach their designated positions.

Since the simulation executes sequentially, the identity of the "central fusion node" rotates (metaphorically) at each iteration. However, as long as the communication graph remains connected, every drone can access the shared covariance matrix and distribute the updated global state estimate to the others. Thus, even though the fusion responsibility is reassigned among drones, the system continues to behave as a centralized estimator.

4.6 Boat positioning and route search

The final phase of the rescue operation involves determining the location of boat and computing an optimal navigable path to the victim, transforming the drones' exploratory data into actionable rescue guidance.

The boat positioning algorithm employs a

multi-criteria scoring system to identify optimal launch locations along the coastline. The placement strategy enforces a minimum safety distance of 8 meters from dangerous obstacles such as reefs and rocks, with fallback thresholds of 4 meters when ideal positions are unavailable. Candidate positions must be located in sea cells adjacent to the target coastline to ensure practical boat deployment, while maintaining an operational distance of approximately 300 meters from the victim.

For route computation, the system implements an optimized A*. The algorithm supports eight-directional movement with the following costs: 1 for cardinal directions and $\sqrt{2}$ for diagonal movements. It incorporates comprehensive collision avoidance mechanisms that check for direct path obstacles and validate diagonal movement corner cases to prevent cutting through impassable areas. The search is guided by a Euclidean distance heuristic that maintains the algorithm's optimality guarantees while ensuring efficient convergence to the solution. Memory-efficient matrix structures enable compact storage of path reconstruction data and state tracking throughout the exploration process. The algorithm systematically processes nodes in order of increasing $f(n)$ values, expanding the most promising paths first while maintaining completeness through exploration of alternatives, with integrated validation including boundary checks, obstacle detection, and connectivity verification to ensure path feasibility.

5 Simulation Results

5.1 Localization with Extended Kalman Filter

Figure 4 illustrates the evolution of the position error for three drones during a mission. Drones 1 and 3 maintain an error bounded to approximately five meters, which is consistent with their relatively small movement uncertainties of $\sigma_{\text{movement}} = 0.5$ m and 1 m, respectively. Drone 2, in contrast, was intentionally assigned a significantly larger motion uncertainty of $\sigma_{\text{movement}} = 5$ m, resulting in considerably higher localization error throughout the mission.

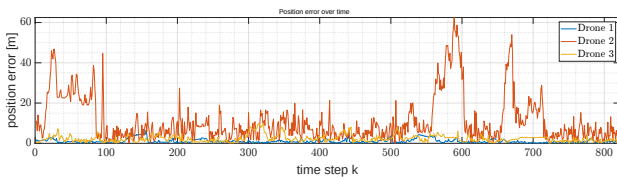


Figure 4. Absolute position error for each drone over time.

From step 720 onward, when the drones begin

navigating back from the target location to the boat, the central fusion algorithm becomes active. This fusion mechanism substantially reduces the absolute position error of Drone 2 and prevents it from increasing further, even as the drones traverse large regions without landmarks. The spike in the measurement noise covariance R , visible in Figure 5 just before step 800, indicates that the drone operated for an extended period in an area devoid of landmarks, forcing the filter to rely purely on odometry. Once central fusion resumes, the relative measurements exchanged among the drones allow the filter to correct accumulated drift effectively. Overall, these results confirm that the central fusion strategy significantly enhances localization accuracy for all agents.

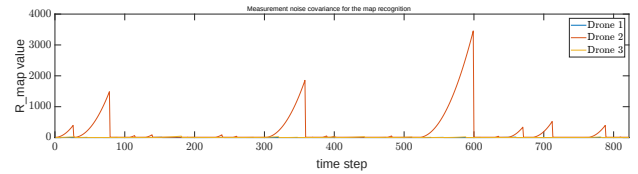


Figure 5. Evolution of the measurement noise covariance R for each drone.

5.2 Consensus Algorithm

Figure 6 shows the local maps produced by the three drones during the mission. All three maps converge to the same occupancy estimate, demonstrating that the drones successfully reached a consensus on the environment. Once consensus is reached, the map of the drone that detected the target is passed to the A* algorithm (see Figure 8).

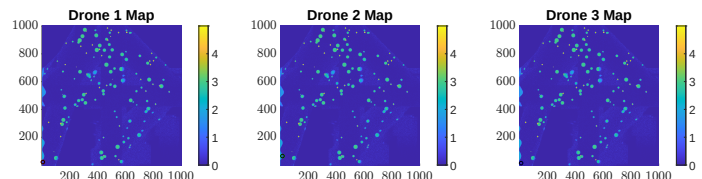


Figure 6. Local occupancy maps generated by three drones during exploration.

Figure 7 highlights the reduction of uncertainty across the maps of all drones. Uncertainty equal zero means the drones explored the whole map and have uncertainty zero for all cells. The sharp drops in uncertainty correspond to communication events, during which drones share their local maps and merge the information contributed by their peers.

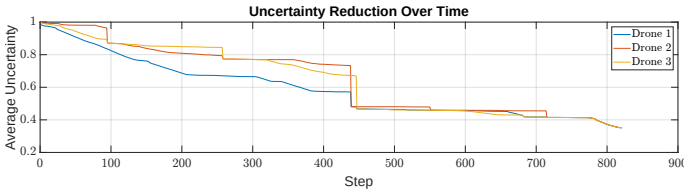


Figure 7. Reduction of map uncertainty for each drone over the duration of the mission. Uncertainty equal zero means the drones explored the whole map and have uncertainty zero for all cells.

5.3 Path to the victim

Our system successfully demonstrates its capability to determine both an optimal boat launch position and compute a safe, efficient navigation path to the victim location. The foundation for successful path planning is established through the comprehensive environmental mapping performed by the drone network. As shown in Figure 8, the drones collectively generate a detailed representation of the operational area, identifying navigable sea regions, hazardous obstacles, and the victim's precise location. This map establishes the complete navigational context necessary for computing the boat's optimal trajectory to the victim. By providing detailed information about obstacle locations and safe passageways, it enables the path planning algorithm to generate a route that maximizes safety while minimizing travel distance, with all navigation decisions grounded in empirically verified environmental data.

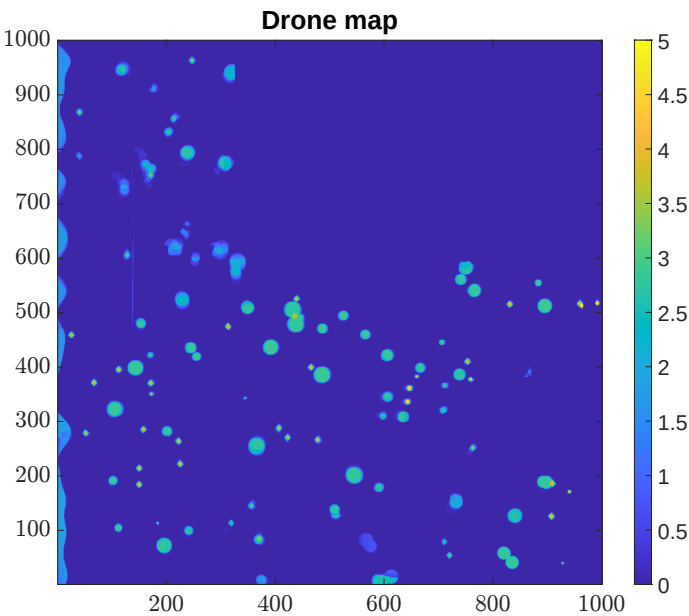


Figure 8. Collaboratively generated environmental map.

For boat positioning, the algorithm employs a weighted multi-criteria system to identify optimal

launch sites, prioritizing safety (50%) through minimum distance thresholds from hazards, operational efficiency (30%) via optimal victim proximity, local accessibility (20%) for maneuvering space, and coastal adjacency (10%) for practical deployment. This approach systematically evaluates 15-25 candidates to select positions that balance safety margins of approximately 7 pixels with operational distances of 300 pixels from the victim.

A key strength of our distributed architecture is the guaranteed map coverage between the boat launch position and the victim location. When any drone locates the victim, it immediately signals the other two drones, which converge to its position. As discussed in Section 3.1, the drones follow a strategic movement pattern that enables them to thoroughly map the entire corridor between the victim and the boat launch site, ensuring that complete environmental information is available for path planning. Consequently, the rescue boat computes an optimal obstacle-avoiding path to the victim, without requiring navigation through unexplored areas.

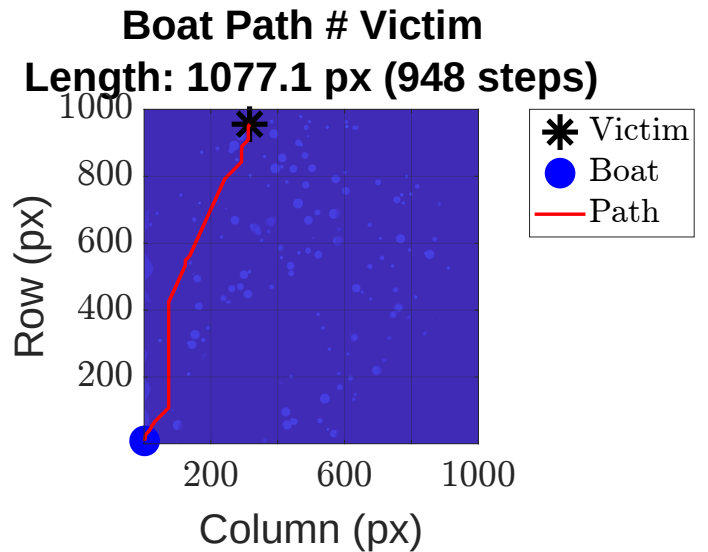


Figure 9. Rescue boat path planning towards the victim location.

The A* pathfinding algorithm demonstrates remarkable efficiency in computing optimal rescue routes through the mapped environment. As illustrated in Figure 9, the algorithm generates smooth, natural navigation paths that maintain safe clearance from all identified obstacles while pursuing the most direct feasible route. The implementation's comprehensive collision avoidance system, including specialized diagonal movement validation, successfully prevents path intersections

with all identified obstacles.

The system's performance validates the effectiveness of the distributed architecture in transforming collaborative environmental sensing into practical rescue guidance, providing rescue teams with both a safe starting point and a reliable navigation plan to reach victims in complex coastal environments.

5.4 Comparison of results

In this section, a comparative analysis of the performance achieved by the different exploration strategies is presented. The comparison focuses on three key metrics to evaluate the efficiency and accuracy of the multi-drone search system:

- **Steps needed to find the victim:** This metric measures the total number of discrete time steps or actions required by the drone swarm to first locate the victim within the environment. A lower number indicates a more efficient and faster search process.
- **Mean Position Error:** This metric quantifies the accuracy of the estimated position of the drones. It is calculated as the RMSE in the Euclidean distance between the true drones coordinates and the coordinates estimated. A lower RMSE signifies higher self localization precision.
- **Mean Position Error before and after the use of the central fusion strategy:** This metric is used to evaluate how much relative measurements between drones can be useful in the self localization before and after the use of a central fusion strategy algorithm.

The evaluation is conducted across a range of swarm sizes, specifically from 2 to 6 drones, to assess scalability. The following exploration methodologies are compared:

- **Random Exploration:** Drones move stochastically with no prior environmental structuring.
- **Uniform Boustrophedon Decomposition (UBD):** The environment is decomposed into cells which are then systematically covered following a back-and-forth (lawnmower) pattern.
- **Random Space Partition (RSP):** The operational area is dynamically partitioned among drones, which then explore their assigned regions.

The subsequent paragraphs and figures will detail the performance of each method against the

forementioned metrics, providing insights into the trade-offs between exploration strategy, swarm size, search speed, and localization accuracy.

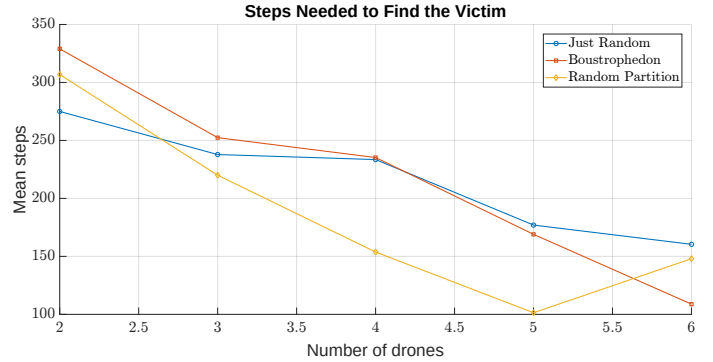


Figure 10. Steps needed to find the target in dependency of the size of the drone swarm and the search algorithm.

The metrics that have been analyzed are the needed time to find the target in dependence of exploration method and swarm size (fig:10), the RMSE in the position, the overall map coverage, and the mean error in localizing in the right position of each placed cell. For each case study 5 simulations were taken to totalize 75 simulations. In the simulation study it was studied the case in which one drone had better proprioceptive and exteroceptive sensors with respect to the others, which had between them the same quality sensors.

Table 1. Average steps to find the victim based on strategy and swarm size

Drones	Random	UBD	RSP
2	275.0	329.0	306.8
3	237.8	252.4	220.0
4	233.5	235.3	153.8
5	177.0	169.0	101.4
6	160.4	108.8	148.0

From table:1 and fig:10 it is possible to notice that a swarm of only two drones the best exploration method is by just exploring the areas the drones didn't find yet without following any strategy. For bigger swarms up to 5, the random partition gives a faster average discovery timing of the target. Then the Boustrophedon area subdivision gives a faster discovery average.

From table:2 and 3, it is possible to see the RMS error in the drone's position respectively, before and after the drones start to move in a connected asset. From the two tables, it is possible to see that when the drones navigate to find the target, the higher the quantity of drones the better is the estimate of the position of the

Table 2. RMS position error (m) before the central fusion algorithm and based on strategy and swarm size

Drones	Random	UBD	RSP
2	3.3	3.5	3.2
3	2.9	2.9	3.0
4	3.0	3.1	2.9
5	2.8	2.7	2.8
6	2.7	2.4	2.5

drones. This can be explained because the higher the number of drones, the higher the probability of a better position estimation using the CI method. Instead, when the drones are in the connected formation, the RMS error in the position remains quite constant through different numbers of drones, which could be explained by the fact that. When few drones are present, the drone that has better sensors has a heavier weight in the position estimation for the other drones, but at the cost of receiving fewer new measurements cause of the smaller area coverage. Instead, when the drones number is higher, even if this effect is not longer present with the same weight, by having more drones more new measurements arrive, but because the drones always have to go to the coast, which is a particular position of the map, because there are no landmarks. The error there tends to grow stronger the higher the number of drones, compared to when only few drones are present.

Table 3. RMS position error (m) when the central fusion algorithm starts being used

Drones	Random	UBD	RSP
2	2.55	2.49	2.07
3	1.82	2.12	1.78
4	1.80	2.31	1.90
5	2.13	1.88	2.10
6	1.89	1.62	2.17

To evaluate how good each exploration algorithm is, another metric that was analyzed is the exploration efficiency, which characterises how fast the drones complete the map (table:4) and with how good precision in the positioning of the obstacles (table:5). From the results shown in the table:4 it can be seen that for few drones the random exploration and the RSP are more efficient than the UBD, and with similar efficiency, then when the drones number raises, the UBD and RSP become more efficient in contrast to the random exploration which doesn't show a clear correlation between drones number and efficiency.

This is because when each drone goes only where no one has yet gone, causes that if two or more drones are near to each other, it happens that instead of dividing the unexplored areas, they all go to the same cells, which reduces the potential benefits of having a higher number of drones.

Table 4. Exploration efficiency, measured as map exploration in % per step

Drones	Random	UBD	RSP
2	0.2034	0.1782	0.2049
3	0.3806	0.2107	0.2270
4	0.3105	0.3833	0.3714
5	0.3857	0.3081	0.4348
6	0.3422	0.6933	0.4933

For the average error in the positioning of the obstacles shown in table:5, no clear difference can be seen between different exploration methods. Instead, for the size of the drone swarm, a small worsening effect in the precision is observed. This again can be explained by the fact that until the drones are few, the positive effect of having a drone with better sensors is more remarkable, and with the increase of more drones, this effect gets diluted.

Table 5. Average error in the positioning of the obstacles measured in m/pixel

Drones	Random	UBD	RSP
2	0.048	0.050	0.050
3	0.047	0.054	0.060
4	0.056	0.050	0.055
5	0.058	0.052	0.060
6	0.060	0.058	0.063

6 Conclusions

The work presented in this report demonstrates the complete development of a distributed control and mapping framework for a small multi-drone system operating in a maritime search-and-rescue scenario. All components of the pipeline were fully implemented and integrated into a coherent simulation environment, allowing the drones to autonomously explore an unknown coastal map, detect the target, align and merge their local representations, and finally provide the rescue team with an optimal route.

The system relies on a lightweight but effective modelling strategy: each drone follows a linear motion model with Gaussian disturbances, while

self-localisation is continuously refined using the Extended Kalman Filter. When landmarks are detected, their measurements enable the reduction of positional drift, and the drones' relative observations are fused through the Covariance Intersection method, ensuring consistency. Environmental perception is handled through a modified WLS algorithm that iteratively refines cell positioning while accounting for both sensing noise and the growth of positional uncertainty over time.

Map sharing is achieved through consensus, using either Metropolis–Hastings weights or row-stochastic matrices, depending on the heterogeneity of sensor quality, to achieve a consensus which converges either to the mean or to the weighted average respectively. When the drones start and find a landmark, through trilateration, they can estimate their position in the same coordinate system, so that a unified Cartesian system is used among the drones. After the target is detected, a centralized fusion stage ensures that all drones contribute to a refined global estimate while maintaining a connected communication graph.

In the final stage, the system identifies a safe and operationally feasible launch point for the rescue boat along the coastline and computes an optimal maritime route using an A* algorithm.

Overall, the report shows that the implemented distributed strategy for the SLAM, effectively enables cooperative localisation, map construction, and route planning under limited connectivity and motion uncertainty. The results show how depending on the quantity of the agents, which strategy among the investigated gives better results, and how using a connected graph type navigation for the drones can improve the overall self localization for all agents.

6.1 Possible Improvements

Although the code is already robust and consistently achieves its final objective, several improvements could further enhance its performance.

First, when a drone acquires a new measurement, it would be beneficial to introduce a mechanism that adjusts all previously observed data according to the magnitude and direction of the new correction. This adjustment should propagate in cascade, assigning the largest correction to the most recent estimates and progressively smaller corrections to those closer to the previously measured landmark.

Second, the map-handling procedure can be improved.

When the map is rotated by angles that are not multiples of 90° , the dimensions should be expanded to prevent values from falling outside the boundaries and being discarded, even if the simulation can often still complete correctly.

A useful improvement would be to take more and new samples for each scenario and for other cases. This is with the aim of further investigating the effects of different standard deviations on the sensors and ensuring that the obtained results for the studied case don't change when a bigger sample size is taken.

Another potential improvement is to allow the drones to explore alternative routes for the rescue team, in cases where a feasible path cannot be found along the trajectory between the target and the starting point.

Finally, the current simulation processes drone operations sequentially. Modifying the system so that all drones operate simultaneously could improve computational efficiency.

References

- [1] Julier, S. J., & Uhlmann, J. K. (2007). Using covariance intersection for SLAM. *Robotics and Autonomous Systems*, 55(1), 3–20. [CrossRef]
- [2] Agostinho, S., Ošep, A., Del Bue, A., & Leal-Taixé, L. (2021). (Just) A Spoonful of Refinements Helps the Registration Error Go Down. *arXiv preprint arXiv:2108.03257*. [PDF]
- [3] Kia, S. S., Rounds, S., & Martinez, S. (2016). Cooperative localization for mobile agents: A recursive decentralized algorithm based on Kalman-filter decoupling. *IEEE Control Systems Magazine*, 36(2), 86–101. [CrossRef]
- [4] Carrillo-Arce, L. C., Nerurkar, E. D., Gordillo, J. L., & Roumeliotis, S. I. (2013). Decentralized multi-robot cooperative localization using covariance intersection. In *Proceedings of the 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems* (pp. 1412–1417). [CrossRef]
- [5] Zhang, Y., Cui, C., & Zhao, Q. (2025). Path Planning of Mobile Robot Based on A Star Algorithm Combining DQN and DWA in Complex Environment. *Appl. Sci.* 2025, 15(8), 4367. [CrossRef]
- [6] Mogens Plessen. Path Planning for Spot Spraying with UAVs Combining TSP and Area Coverages. *Applied Soft Computing and Communication Networks*, 2025, 15(8), 4367. [CrossRef]

[Link for the Github project repository]